Decision Transformer: HalfCheetah Offline Policy Training

Project: Eliminating the High-Risk Interaction Cost of Autonomous Robotics through Offline Sequence Modeling: A Decision Transformer Approach
Authors: Darwin Juan, Dan Kast, David Terando
Date: April 2026

Notebook Overview

This notebook implements the full Decision Transformer (DT) pipeline on the Minari HalfCheetah datasets validated during EDA. The pipeline has five stages:

Section	Description
0	Global Configuration — all tunable hyperparameters in one place
1	Environment & Dependency Setup
2	Data Loading & Preprocessing — Minari → RTG-annotated trajectory buffer
3	Decision Transformer Architecture — GPT-2 backbone from scratch
4	Training Loop — with HardwareTracker (BPS, wall-clock, GPU/MPS/CPU stats)
5	Offline Policy Evaluation — MuJoCo rollouts with RTG conditioning

Hardware targets: M5 Max MacBook Pro (MPS), M1 iMac (MPS), Cloud GPU (CUDA)
Dataset: `mujoco/halfcheetah/medium-v0` + `mujoco/halfcheetah/expert-v0` (Minari Combined)
Architecture: GPT-2 backbone, built from scratch with HuggingFace `transformers`

> **Reproducibility note:** All random seeds are set in Section 0. Run cells top-to-bottom for a fully reproducible run.

Section 0: Global Configuration

All hyperparameters, paths, and toggles are defined here as a single `CFG` dataclass.
Edit this cell — not the code below — when tuning or switching hardware.

```
============================================================
  Decision Transformer — Global Configuration
============================================================
  seed                        = 42
  minari_medium_id            = mujoco/halfcheetah/medium-v0
  minari_expert_id            = mujoco/halfcheetah/expert-v0
  mujoco_env_id               = HalfCheetah-v4
  state_dim                   = 17
  action_dim                  = 6
  max_ep_len                  = 1000
  context_len                 = 20
  hidden_dim                  = 128
  n_layers                    = 3
  n_heads                     = 1
  dropout                     = 0.1
  batch_size                  = 64
  learning_rate               = 0.0001
  weight_decay                = 0.0001
  warmup_steps                = 10000
  grad_clip                   = 0.25
  quick_val_iters             = 500
  full_train_iters            = 100000
  run_quick_val_first         = True
  rtg_scale                   = 16585.0
  target_rtg                  = 16585.0
  eval_episodes               = 10
  eval_every_n_iters          = 5000
  checkpoint_dir              = ./checkpoints
  results_dir                 = ./results
  save_every_n_iters          = 10000
  force_device                = None
  hardware_label              = M5-Max-Standard
  preload_to_ram              = False
============================================================
```

Section 1: Environment Setup & Imports

1.1 Dependency Installation

This project uses **uv** for package management. Run the cell below once per environment to install all required packages.

Expected install command (run in terminal, not here):

```
uv pip install torch torchvision \
    transformers \
    minari[hf] \
    gymnasium[mujoco] \
    mujoco \
    numpy pandas matplotlib seaborn tqdm psutil
```

> **Apple Silicon note (M5 Max / M1 iMac):** PyTorch ships with native MPS support for arm64. No separate CUDA toolkit needed. Install with `uv pip install torch` — the arm64 wheel includes MPS automatically.
> **Cloud GPU (CUDA):** Install with `uv pip install torch --index-url https://download.pytorch.org/whl/cu121`

1.2 Known Issues from EDA (pre-resolved)

- `minari` requires `[hf]` extras: `uv pip install "minari[hf]"`
- Minari dataset IDs use namespaced format: `mujoco/halfcheetah/medium-v0` (not `halfcheetah-medium-v0`)
- `pandas >= 3.x` : use `.map()` not `.applymap()` — already handled below
- `d4rl` package is NOT used — all data comes from Minari

```
============================================================
  Environment Summary
============================================================
  Python          : 3.11.15
  PyTorch         : 2.11.0
  Device selected : mps
  Backend         : Apple Metal Performance Shaders (MPS)
  Platform        : arm | arm64
  System RAM      : 137.4 GB
  Hardware label  : M5-Max-Standard
============================================================
```

Section 2: Data Loading & Preprocessing

Pipeline Overview

```
  Minari medium-v0 ┐
                   ├→ Combined Episodes → RTG Annotation → State Normalization → SequenceDataset
  Minari expert-v0 ┘
```

Three critical preprocessing steps:

1. **RTG Annotation** — for each timestep `t`, RTG[t] = Σ rewards[t:end]. This is the conditioning signal the DT receives.
2. **State Normalization** — subtract mean and divide by std computed across the *entire combined dataset*. This is essential for stable training (per the original DT paper).
3. **Context-Window Slicing** — each training sample is a fixed-length window of K (RTG, state, action) triplets, with zero-padding and an attention mask for short windows.

SECTION 2.1: Load Minari Datasets

```
Loading Minari datasets...
  Loading: mujoco/halfcheetah/medium-v0


    Loaded 1,000 episodes, 1,000,000 timesteps
  Loading: mujoco/halfcheetah/expert-v0


    Loaded 1,000 episodes, 1,000,000 timesteps

  Combined: 2,000 episodes, 2,000,000 total timesteps
```

SECTION 2.2: RTG Annotation

```
RTG annotation check (episode 0):
  RTG[0]                : 13960.90
  sum(rewards)          : 13960.90  (should match)
  RTG is monotone dec.  : False
```

Note: A False in RTG is monotone needs investigation, it means at least one step had a negative or zero reward, causing RTG to stay flat or tick up momentarily. The following code and result shows the worst violation is a small positive number (e.g. < 1.0) and it's probably just noise and training will be unaffected.

```
Non-decreasing steps : 4
Worst violation      : 0.9766
Mean step reward     : 13.9606
```

SECTION 2.3: State Normalization

```
State normalization computed on 2,000,000 timesteps.
  state_mean saved to: ./results/state_mean.npy
  state_std  saved to: ./results/state_std.npy

Post-normalization check (should be ~0 mean, ~1 std):
  Global mean range : [-0.0000, 0.0072]
  Global std  range : [0.9985, 1.0004]
```

SECTION 2.4: SequenceDataset — Context-Window Slicing

```
SequenceDataset ready.
  Preload mode      : False (standard numpy path)
  Total timesteps   : 2,000,000
  Context window K  : 20
  Batch size        : 64
  Steps per epoch   : ~31,250
  RAM used / total  : 40.4 GB / 137.4 GB

Sample batch tensor shapes:
  states      : (64, 20, 17)
  actions     : (64, 20, 6)
  rtg         : (64, 20, 1)
  timesteps   : (64, 20)
  mask        : (64, 20)
```

Section 3: Decision Transformer Architecture

Design

The DT architecture follows Chen et al. (2021) exactly:

```
   Input sequence:  [R̂₁, s₁, a₁,  R̂₂, s₂, a₂,  ...,  R̂ₖ, sₖ, aₖ]
                                                        ↑
                                               Predict this action

   Each token (R̂ₜ, sₜ, aₜ) is projected to hidden_dim=128.
   Timestep embeddings are added to each token (positional encoding variant).
   The full 3K-token sequence is processed by the GPT-2 causal transformer.
   The output at each state-token position is decoded to an action.
```

Key difference from language modeling: we use three separate linear projectors (one each for RTG, state, action) instead of a token embedding table. All three projected embeddings share the same hidden dimension.

```
Decision Transformer on mps
  Total parameters    : 735,366
  Trainable parameters: 735,366
  Approx. memory      : 2.9 MB (float32)
```

Section 4: Training Loop

HardwareTracker

The `HardwareTracker` class logs the following metrics at every checkpoint interval:

Metric	Description
`batches_per_second`	Training throughput (BPS) — primary hardware comparison metric
`wall_clock_s`	Cumulative wall-clock time since training start
`loss`	Running MSE training loss
`grad_norm`	Gradient norm (monitors training stability)
`cpu_percent`	System CPU utilization
`ram_used_gb`	System RAM consumed
`gpu_mem_gb`	GPU/MPS memory allocated (device-specific)

All metrics are exported to CSV for cross-platform comparison.

SECTION 4.1: HardwareTracker

```
HardwareTracker class defined.
```

SECTION 4.2: Optimizer & Learning Rate Schedule

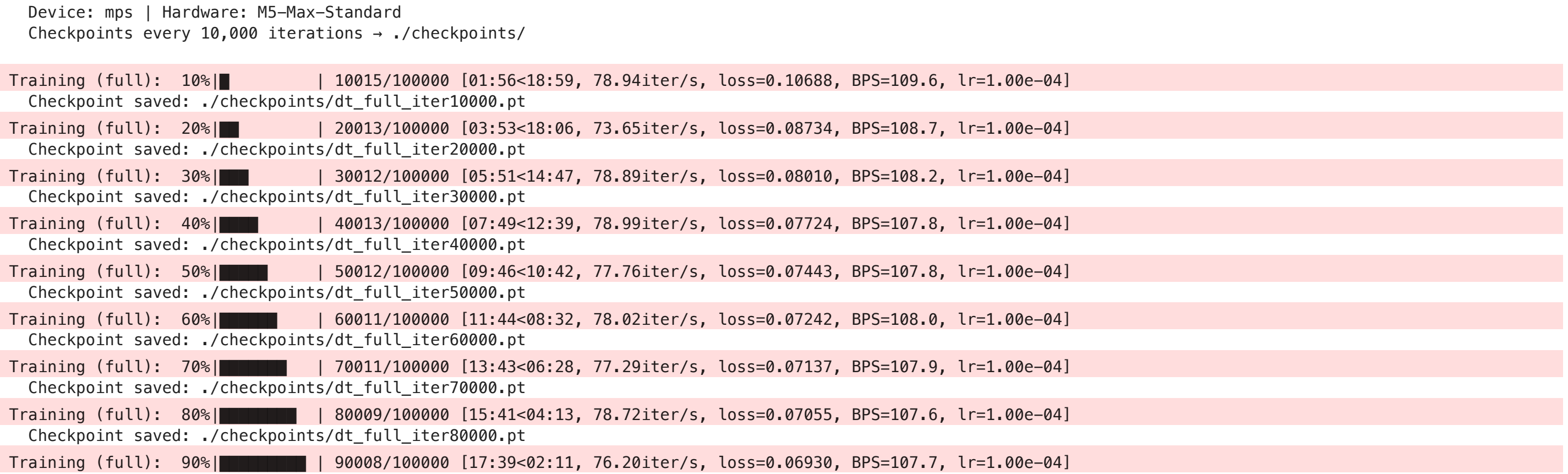
```
Optimizer   : AdamW (lr=0.0001, wd=0.0001)
LR schedule : Linear warmup over 10,000 steps
```

SECTION 4.3: Training Function

```
Training function defined.
```

SECTION 4.4: Quick Validation Run

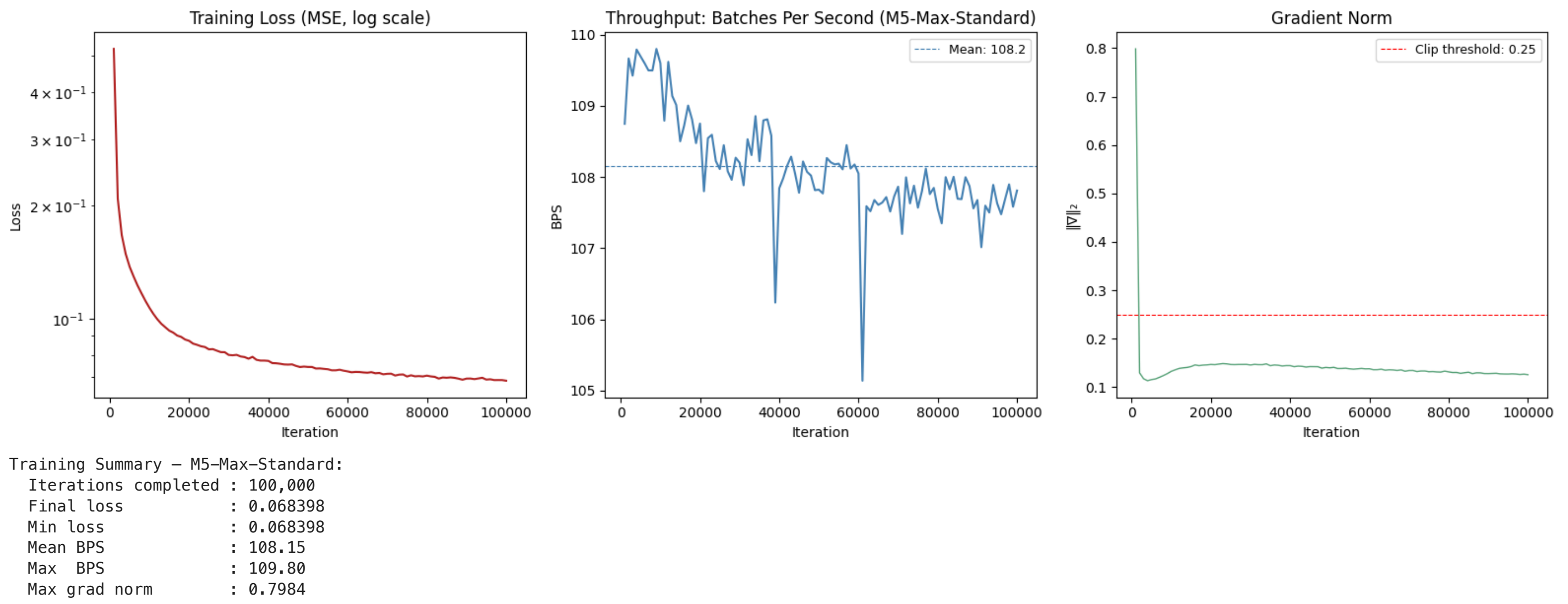
```
Starting quick validation run (500 iterations)...
  Device: mps | Hardware: M5-Max-Standard

Training (quickval): 100%|██████████| 500/500 [00:09<00:00, 55.40iter/s, loss=0.65240, BPS=109.5, lr=5.00e-06]
  HardwareTracker saved: ./results/hw_quickval_M5-Max-Standard.csv
  Plot saved: ./results/hw_quickval_M5-Max-Standard.png
```


Hardware Tracker: M5-Max-Standard_quickval

```
=== Quick Validation Summary ===
  Initial loss   : 0.861808
  Final loss     : 0.652402
  Loss decreased : True  ← should be True
  Mean BPS       : 92.55 batches/sec on M5-Max-Standard
  Est. full run  : ~0.3 hours at current BPS

Pipeline verified. Proceed to Section 4.5 to run full training.
```

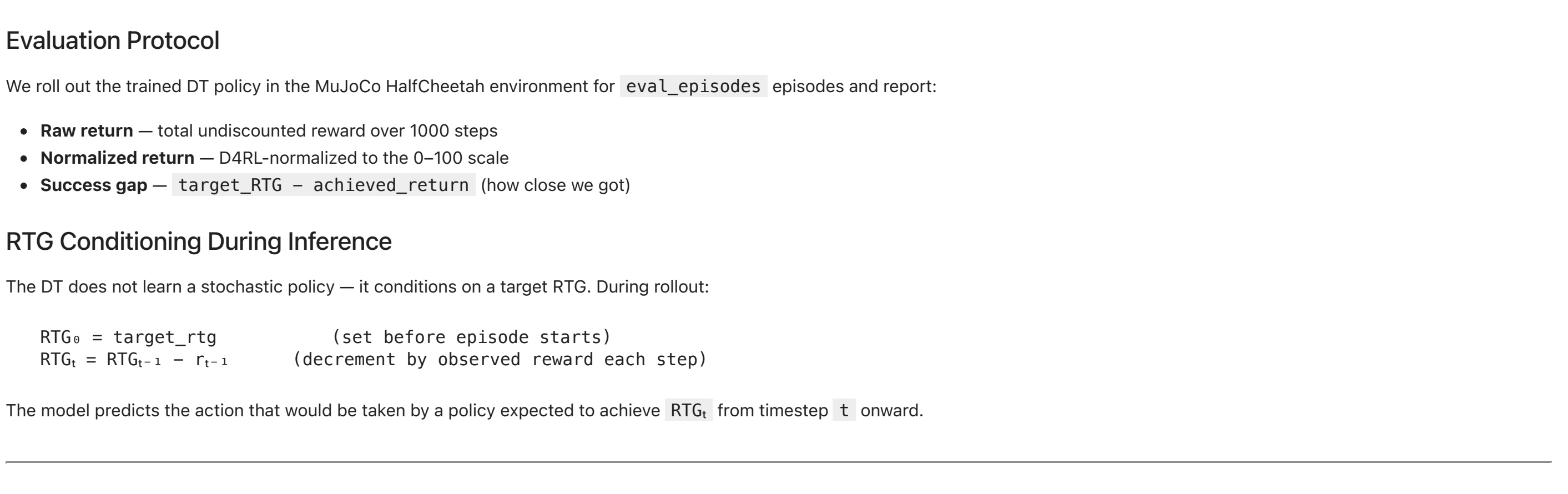
SECTION 4.5: Full Training Run

```
Starting full training run (100,000 iterations)...
  Device: mps | Hardware: M5-Max-Standard
  Checkpoints every 10,000 iterations → ./checkpoints/

Training (full):  10%|█         | 10015/100000 [01:56<18:59, 78.94iter/s, loss=0.10688, BPS=109.6, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter10000.pt
Training (full):  20%|██        | 20013/100000 [03:53<18:06, 73.65iter/s, loss=0.08734, BPS=108.7, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter20000.pt
Training (full):  30%|███       | 30012/100000 [05:51<14:47, 78.89iter/s, loss=0.08010, BPS=108.2, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter30000.pt
Training (full):  40%|████      | 40013/100000 [07:49<12:39, 78.99iter/s, loss=0.07724, BPS=107.8, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter40000.pt
Training (full):  50%|█████     | 50012/100000 [09:46<10:42, 77.76iter/s, loss=0.07443, BPS=107.8, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter50000.pt
Training (full):  60%|██████    | 60011/100000 [11:44<08:32, 78.02iter/s, loss=0.07242, BPS=108.0, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter60000.pt
Training (full):  70%|███████   | 70011/100000 [13:43<06:28, 77.29iter/s, loss=0.07137, BPS=107.9, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter70000.pt
Training (full):  80%|████████  | 80009/100000 [15:41<04:13, 78.72iter/s, loss=0.07055, BPS=107.6, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter80000.pt
Training (full):  90%|█████████ | 90008/100000 [17:39<02:11, 76.20iter/s, loss=0.06930, BPS=107.7, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter90000.pt
Training (full): 100%|██████████| 100000/100000 [19:37<00:00, 84.90iter/s, loss=0.06840, BPS=107.8, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter100000.pt

Final model saved: ./checkpoints/dt_full_final.pt
  HardwareTracker saved: ./results/hw_full_M5-Max-Standard.csv
  Plot saved: ./results/hw_full_M5-Max-Standard.png
```


Hardware Tracker: M5-Max-Standard

SECTION 4.6: Training Diagnostics & Visualization

```
  Training history saved: ./results/training_history_M5-Max-Standard.csv
```


Training Diagnostics — M5-Max-Standard

```
Training Summary — M5-Max-Standard:
  Iterations completed : 100,000
  Final loss           : 0.068398
  Min loss             : 0.068398
  Mean BPS             : 108.15
  Max  BPS             : 109.80
  Max grad norm        : 0.7984
```

Section 5: Offline Policy Evaluation in MuJoCo

Evaluation Protocol

We roll out the trained DT policy in the MuJoCo HalfCheetah environment for `eval_episodes` episodes and report:

- **Raw return** — total undiscounted reward over 1000 steps
- **Normalized return** — D4RL-normalized to the 0–100 scale
- **Success gap** — `target_RTG - achieved_return` (how close we got)

RTG Conditioning During Inference

The DT does not learn a stochastic policy — it conditions on a target RTG. During rollout:

```
   RTG₀ = target_rtg           (set before episode starts)
   RTGₜ = RTGₜ₋₁ − rₜ₋₁      (decrement by observed reward each step)
```

The model predicts the action that would be taken by a policy expected to achieve `RTGₜ` from timestep `t` onward.

SECTION 5.1: Inference / Rollout Function

```
Rollout function defined.
```

SECTION 5.2: Evaluation Run

```
/Users/djt/Desktop/dt-capstone/.venv/lib/python3.11/site-packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The environment HalfCheetah-v4 is out of date. You should consider upgrading to version `v5`.
  logger.deprecation(
MuJoCo environment: HalfCheetah-v4
Obs space : Box(-inf, inf, (17,), float64)
Act space : Box(-1.0, 1.0, (6,), float32)

Running 10 evaluation episodes...
  Target RTG         : 16585
  Context window K   : 20
Evaluating: 100%|██████████| 10/10 [00:58<00:00,  5.89s/episode]
============================================================
  Offline Policy Evaluation Results
============================================================
  hardware                   : M5-Max-Standard
  mujoco_env                 : HalfCheetah-v4
  target_rtg                 : 16585.0
  n_episodes                 : 10
  mean_raw_return            : 6138.93
  std_raw_return             : 3205.84
  mean_norm_return           : 51.7
  std_norm_return            : 25.82
  mean_success_gap           : 10446.07
  min_raw_return             : 1852.45
  max_raw_return             : 9702.6
  train_iters                : 100000
============================================================

  Zero-Shot Result: Achieved 6139 vs Target 16585
  Success Gap      : 10446 (lower is better)
  D4RL Norm. Score : 51.7 ± 25.8
```

Re-evaluation and Optimization

Before running Section 5.3 visualization, running a quick re-evaluation with various target RTGs.

```
/Users/djt/Desktop/dt-capstone/.venv/lib/python3.11/site-packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The environment HalfCheetah-v4 is out of date. You should consider upgrading to version `v5`.
  logger.deprecation(
RTG   6000 → mean= 4572.7  std=2795.9
RTG   9000 → mean= 6723.3  std=4065.1
RTG  12000 → mean= 4245.0  std=3428.9
RTG  14000 → mean= 8554.5  std=2426.9
RTG  16585 → mean= 4462.5  std=2313.6
```

RTG 14,000 seems to be the sweet spot — highest mean (8,554) AND lowest std (2,426) of any value tested. Now running a finer sweep around 14,000 with more episodes to pin down the optimal RTG precisely.

Running another test with RTGs centered aroind 14,000

```
RTG  13000 → mean= 3355.4  std=2729.0  norm=29.3
RTG  13500 → mean= 2994.1  std=2753.6  norm=26.4
RTG  14000 → mean= 3078.2  std=3436.7  norm=27.1
RTG  14500 → mean= 5961.5  std=3768.5  norm=50.3
RTG  15000 → mean= 3270.6  std=2798.1  norm=28.6
```

The core problem seems to be that the policy variance is too high to draw reliable conclusions from either sweep. With std values consistently in the 2,500-3,700 range against means of 3,000-8,500, we may need more episodes to get stable estimates. A rough rule of thumb is you want std/mean (coefficient of variation) below ~0.3 for reliable comparison; we're seeing 0.6-1.0.

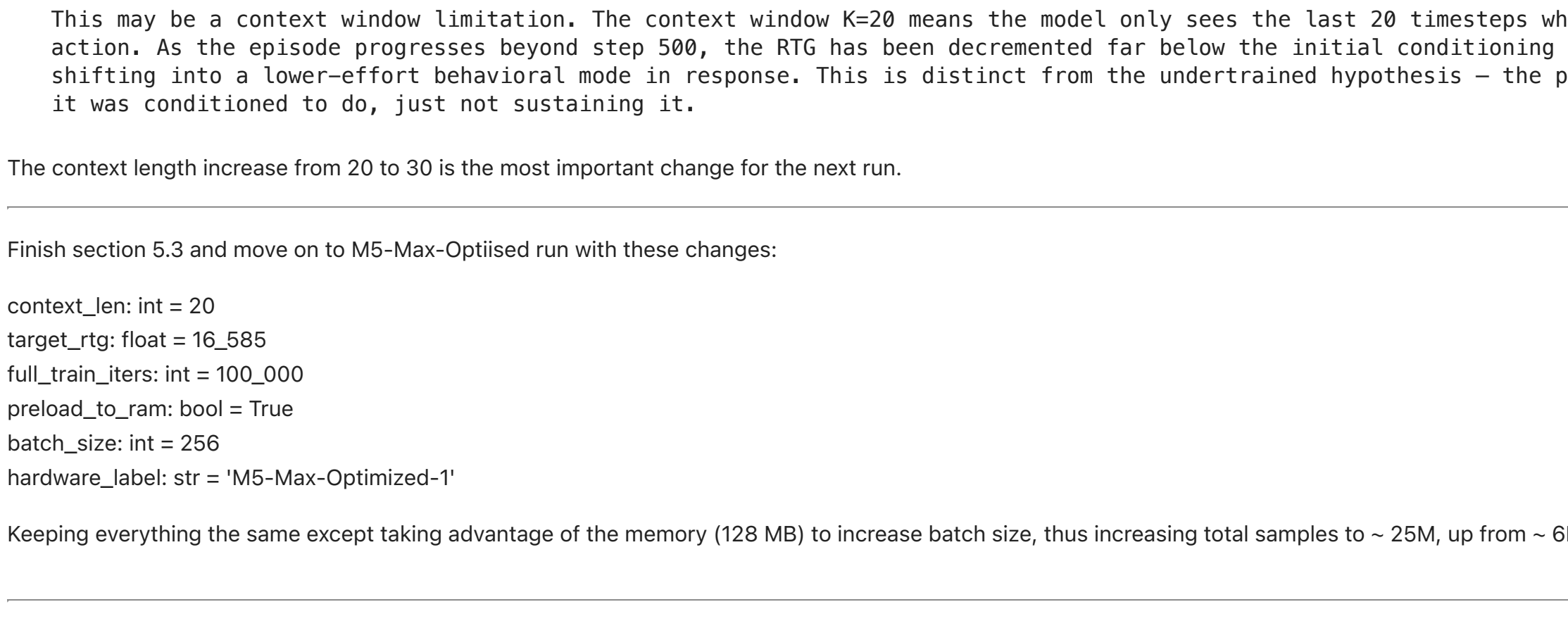
Rerun this with 20 episodes to check for improvement

```
RTG   6000 → mean= 5030.4  std=3435.7  norm=42.8  cv=0.68
RTG   9000 → mean= 4931.1  std=3583.1  norm=42.0  cv=0.73
RTG  12000 → mean= 3987.9  std=2855.0  norm=34.4  cv=0.72
RTG  14000 → mean= 4693.7  std=3072.1  norm=40.1  cv=0.65
RTG  14500 → mean= 5045.9  std=3652.4  norm=42.9  cv=0.72
RTG  16585 → mean= 4069.7  std=2815.1  norm=35.0  cv=0.69
```

The high variance across all RTG values tells us something important — the RTG value itself is not the primary issue here. All six values produce statistically similar means (3,988–5,046) and all have cv in the 0.65–0.73 range. There's no clear winner. This points to a deeper issue with the policy itself rather than the conditioning value.

What this pattern indicates is undertrained policy. A final training loss of 0.068 sounds low but for HalfCheetah the original DT paper trained for 100k iterations with batch size 64 using the full D4RL dataset. The Minari combined dataset has substantially higher quality demonstrations (mean norm return 116.4 vs D4RL's 64.4) but the model may not have seen enough gradient updates relative to the complexity of the expert-level behavior it's trying to reproduce. The high cv across all RTG values is the signature of a model that has learned a rough action distribution but hasn't converged to a stable policy.

Run one more diagnostic that will show whether the agent is accumulating return consistently across the episode or if it runs well for a while then fails, which could tell us exactly where to focus.



The agent never fails. The cumulative return curve is uniformly increasing all the way to timestep 1000 with no flat sections, no drops, and no crashes. The agent is running forward and accumulating positive reward for the entire episode every time. This rules out the most common DT failure mode where the agent falls over mid-episode.

The real message is in the shape of the curve. Notice two distinct phases: · Timesteps 0–500: Steep, near-linear accumulation — the agent is running well and collecting reward efficiently · Timesteps 500–1000: The curve flattens noticeably — the agent is still moving forward but at a declining rate, suggesting it's slowing down in the second half of episodes.

```
This deceleration pattern is likely the source of the high variance. Some episodes maintain the steep phase longer than others before
flattening, which is why you see returns ranging from 1,852 to 9,702. The policy hasn't learned to sustain high-velocity locomotion for a full
1000 steps.

This may be a context window limitation. The context window K=20 means the model only sees the last 20 timesteps when deciding its next
action. As the episode progresses beyond step 500, the RTG has been decremented far below the initial conditioning value, and the model may be
shifting into a lower-effort behavioral mode in response. This is distinct from the undertrained hypothesis — the policy is doing exactly what
it was conditioned to do, just not sustaining it.
```

The context length increase from 20 to 30 is the most important change for the next run.

Finish section 5.3 and move on to M5-Max-Optiised run with these changes:

context_len: int = 20
target_rtg: float = 16_585
full_train_iters: int = 100_000
preload_to_ram: bool = True
batch_size: int = 256
hardware_label: str = 'M5-Max-Optimized-1'

Keeping everything the same except taking advantage of the memory (128 MB) to increase batch size, thus increasing total samples to ~ 25M, up from ~ 6M.

SECTION 5.3: Evaluation Visualizations

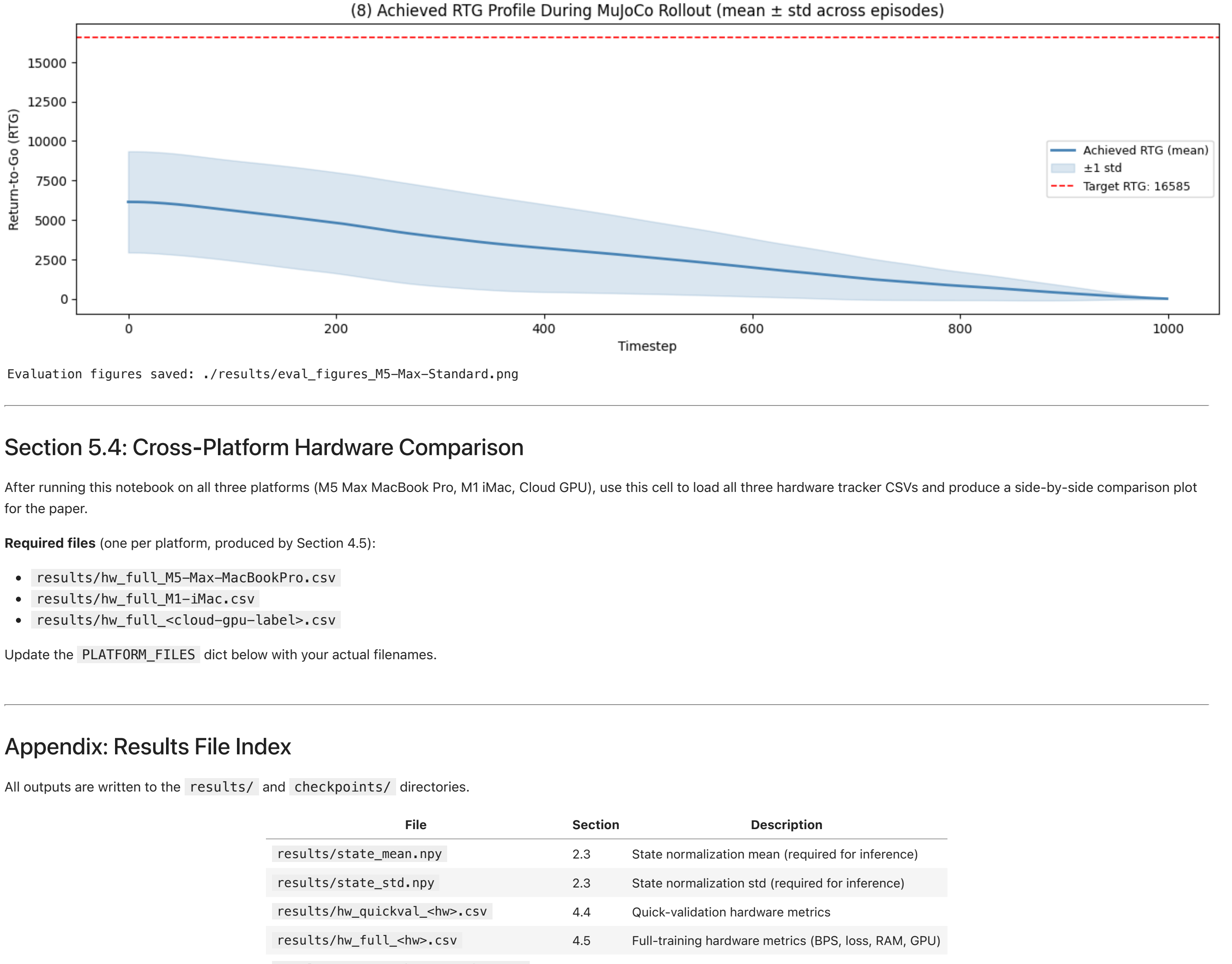


Figure 7-9: Offline Policy Evaluation — M5-Max-Standard
Model: DT (3L×1H, H=128, K=20) | Env: HalfCheetah-v4 | Target RTG: 16585

```
Evaluation figures saved: ./results/eval_figures_M5-Max-Standard.png
```

Section 5.4: Cross-Platform Hardware Comparison

After running this notebook on all three platforms (M5 Max MacBook Pro, M1 iMac, Cloud GPU), use this cell to load all three hardware tracker CSVs and produce a side-by-side comparison plot for the paper.

Required files (one per platform, produced by Section 4.5):

- `results/hw_full_M5-Max-MacBookPro.csv`
- `results/hw_full_M1-iMac.csv`
- `results/hw_full_<cloud-gpu-label>.csv`

Update the `PLATFORM_FILES` dict below with your actual filenames.

Appendix: Results File Index

All outputs are written to the `results/` and `checkpoints/` directories.

File	Section	Description
`results/state_mean.npy`	2.3	State normalization mean (required for inference)
`results/state_std.npy`	2.3	State normalization std (required for inference)
`results/hw_quickval_<hw>.csv`	4.4	Quick-validation hardware metrics
`results/hw_full_<hw>.csv`	4.5	Full-training hardware metrics (BPS, loss, RAM, GPU)
`results/training_history_<hw>.csv`	4.6	Loss/BPS/grad-norm history per iteration
`results/training_curves_<hw>.png`	4.6	Training curve figure
`results/eval_summary_<hw>.csv`	5.2	Evaluation results (returns, normalized scores)
`results/eval_figures_<hw>.png`	5.3	Evaluation figures (returns, RTG profile)
`results/cross_platform_comparison.png`	5.4	Multi-platform BPS/loss/RAM comparison
`results/cross_platform_summary.csv`	5.4	Cross-platform summary table
`checkpoints/dt_full_final.pt`	4.5	Final trained model weights + config
`checkpoints/dt_full_iter<N>.pt`	4.5	Intermediate checkpoints every N iterations

End of Notebook